SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*
Assessment Tool Weightage: 30%

Reverse Engineering Task

Code of Conduct:

I Soorya S / 192311060 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SooryaS

1. ATM Withdrawal Module – Black Box Testing

Question:
A partially tested ATM withdrawal module is provided where only input-output behavior is known. Reverse engineer the system by analyzing observed behavior and reconstructing the test case design strategy. Identify functional components, equivalence classes, boundary conditions, and improve the test cases.

Answer:

1. Understanding the System

The ATM withdrawal module allows users to withdraw money after validating their account, PIN, balance, and withdrawal limits. It displays success or error messages based on the transaction result.

2. System Decomposition

- Authentication: Validates the ATM card and PIN.
- Balance Validation: Checks whether sufficient funds are available.
- Limit Validation: Checks daily and per-transaction withdrawal limits.
- Transaction Processing: Deducts the withdrawal amount.
- Output Handling: Displays confirmation or error messages.

3. Equivalence Class Partitioning (ECP)

Input	Valid Class	Invalid Class
PIN	Correct PIN	Incorrect PIN
Amount	Positive permitted amount	Zero, negative, or invalid amount
Balance	Sufficient balance	Insufficient balance
Daily limit	Within remaining limit	Exceeds remaining limit

4. Boundary Value Analysis (BVA)

Assume the minimum withdrawal is ₹100, the maximum per transaction is ₹25,000, and the available balance is ₹10,000.

Test Case	Input	Expected Result
TC1	₹0	Reject withdrawal

TC2	₹100	Accept if other conditions pass
TC3	₹101	Accept if other conditions pass
TC4	₹10,000	Accept if other conditions pass
TC5	₹10,001	Reject due to insufficient balance
TC6	₹25,000	Reject due to insufficient balance

5. Improved Test Strategy

Test valid and invalid PINs, insufficient balances, exceeded daily limits, invalid denominations, network failures, and repeated withdrawals. Verify that failed transactions do not deduct money and successful transactions update the balance correctly.

Conclusion: ECP reduces redundant tests, while BVA detects errors near limits. Combining both techniques improves the reliability and security of the ATM module.

---

2. Control Flow Graph and White Box Testing

Question:
A software module's execution traces and code snippets are provided without test documentation. Construct a control flow graph, identify decisions, loops, and independent paths, evaluate coverage, and reconstruct a complete path-testing strategy.

Answer:

1. Understanding the System

White box testing examines the internal program structure, decisions, loops, and execution paths. A Control Flow Graph (CFG) represents the flow of control between program statements.

2. Sample Program

```
def check_number(n):
    if n > 0:
        if n % 2 == 0:
            print("Positive even")
        else:
            print("Positive odd")
    else:
```

```
    print("Non-positive")
```

3. Control Flow Graph

Start → Check whether n > 0.

- If true, check whether n is divisible by 2.
  - If true, display "Positive even".
  - If false, display "Positive odd".
- If false, display "Non-positive".

All branches then proceed to the End node.

4. Independent Execution Paths

Path	Input	Expected Output
P1	4	Positive even
P2	3	Positive odd
P3	0	Non-positive
P4	-2	Non-positive

There are three feasible independent paths in this example. Zero and negative values follow the same control-flow branch, although both are useful test inputs.

5. Coverage Evaluation

- Statement Coverage: Executes every statement at least once.
- Branch Coverage: Executes both true and false outcomes of every decision.
- Path Coverage: Tests feasible execution paths through the program.
- Loop Coverage: Tests zero, one, and multiple iterations where loops exist.

6. Improved Strategy

Execute tests for positive even, positive odd, zero, and negative inputs. For larger programs, identify all decision nodes, calculate cyclomatic complexity, and select independent paths. Add loop-boundary tests wherever loops exist.

Conclusion: CFG-based white box testing reveals untested branches and execution paths, improving logic coverage and defect detection.

3. Web Login and Transaction System – Integrated Testing

Question:
A web application login and transaction system has unknown testing techniques. Analyze defect logs, identify likely black box and white box methods, find coverage gaps, and reconstruct a strategy using ECP, BVA, and path testing.

Answer:

1. Understanding the System

The application authenticates users and allows authorized users to perform financial transactions. Defects may occur in authentication, input validation, transaction processing, and database updates.

2. System Decomposition

- Login: Validates username and password.
- Session Management: Maintains authenticated sessions.
- Authorization: Checks user permissions.
- Transaction Module: Validates amount and available balance.
- Database: Stores transaction details and updated balances.
- Error Handling: Displays appropriate messages and handles failures.

3. Likely Testing Techniques

- ECP: Groups valid and invalid usernames, passwords, and transaction amounts.
- BVA: Checks minimum and maximum input limits.
- Branch Testing: Examines true and false outcomes of authentication and balance decisions.
- Path Testing: Verifies complete login-to-transaction execution paths.
- Negative Testing: Checks unauthorized access and invalid transactions.

These techniques can be inferred from defect patterns, but logs alone cannot prove exactly which methods were originally used.

4. Test Cases

Test Case	Input or Condition	Expected Result
TC1	Valid credentials	Login successful
TC2	Incorrect password	Login rejected
TC3	Empty username	Validation error
TC4	Transaction amount ₹0	Rejected
TC5	Amount within balance and limit	Transaction successful

TC6	Amount exceeds balance	Rejected
TC7	Session expires during transaction	Securely reject or recover
TC8	Database failure	No incorrect debit; safe error

5. Coverage Gaps

Testing only successful logins misses invalid credentials, session expiration, unauthorized transactions, and repeated submissions. Testing only outputs may also miss incorrect internal branches and database updates.

6. Reconstructed Testing Strategy

First, use ECP to group input conditions. Next, apply BVA to minimum and maximum limits. Finally, construct a CFG for authentication and transaction processing, then test success, rejection, session timeout, and database-failure paths. Verify that each transaction is recorded exactly once.

Conclusion: Combining black box and white box testing validates both external behavior and internal program logic, reducing security and transaction-related defects.

---

4. Inventory Management System – Stock Updates

Question:
A legacy inventory management system provides successful and failed transaction logs for stock updates and low-stock alerts. Reverse engineer its business logic, identify hidden boundaries and equivalence partitions, and reconstruct the black box test cases.

Answer:

1. Understanding the System

The inventory system updates stock quantities when products are received or sold. It also generates low-stock alerts when the quantity falls below a predefined threshold.

2. System Decomposition

- Product Identification: Checks whether the product exists.
- Stock Input Validation: Validates quantity and transaction type.
- Stock Update: Adds received stock or subtracts sold stock.
- Threshold Validation: Compares current stock with the reorder level.
- Alert Generation: Sends an alert when stock is low.

3. Equivalence Partitions

Input	Valid Class	Invalid Class
Product ID	Existing product	Unknown product
Quantity	Positive integer	Zero, negative, or invalid value
Stock level	Sufficient stock	Insufficient stock for sale
Threshold	Stock above threshold	Stock at or below threshold, depending on the specified rule

4. Boundary Value Analysis

Assume the low-stock threshold is 10 units.

Test Case	Stock After Update	Expected Result
TC1	9	Low-stock alert
TC2	10	Alert if rule is stock ≤ 10
TC3	11	No low-stock alert
TC4	0	Out-of-stock status
TC5	-1	Reject invalid stock quantity

5. Reconstructed Test Cases

Test stock receipt, product sales, unknown product IDs, duplicate updates, invalid quantities, concurrent stock updates, and alert generation. Verify that stock is never reduced below zero unless negative inventory is explicitly permitted.

6. Justification

The boundary tests identify errors around the reorder threshold. Equivalence partitioning reduces redundant inputs, while transaction tests verify correct stock calculations and alerts.

Conclusion: A combination of ECP and BVA improves inventory accuracy and prevents incorrect stock updates and missed low-stock alerts.

---

5. Library Management Module – White Box Path Testing

Question:
A compiled .exe file of a library management module and CPU/memory execution profiles are provided. Map execution spikes to potential loops and decision clusters, construct a conceptual CFG, and develop path tests that target resource-intensive paths.

Answer:

1. Understanding the System

The library management module manages book searches, issue and return operations, member records, and overdue calculations. CPU or memory spikes may indicate repeated searches, nested loops, large data structures, or inefficient database operations.

2. System Decomposition

- Book Search: Finds books using IDs, titles, or authors.
- Member Validation: Checks membership status and borrowing limits.
- Issue/Return Processing: Updates book availability.
- Overdue Calculation: Computes fines and due dates.
- Database Operations: Reads and updates book and member records.

3. Conceptual Control Flow Graph

Start → Load book records → Check whether the book exists.

- If the book exists, validate the member and execute the issue/return process.
- If the book does not exist, display an error.
- Both branches proceed to the End node.

This is a conceptual CFG. The actual executable's internal paths cannot be confirmed from resource profiles alone.

4. Reconstructed Path Tests

Test Case	Condition	Expected Result
TC1	Book exists, member valid	Issue successful
TC2	Book does not exist	Error displayed
TC3	Member invalid	Issue rejected
TC4	Borrowing limit reached	Issue rejected
TC5	Large book collection	Search completes efficiently
TC6	Repeated issue/return operations	Correct records with no resource leak

5. Resource Analysis

Use profiling tools to measure CPU usage, memory consumption, and execution time for each workload. Test empty, small, and large datasets, repeated operations, and invalid requests. If spikes occur in repeated searches, optimize the search algorithm or database indexing.

6. Stability Evaluation

Stress testing identifies excessive memory use, slow responses, and possible resource leaks. White box path testing validates logic, while performance profiling helps locate expensive execution regions.

Conclusion: Combining path testing, stress testing, and profiling improves stability and efficiency. A compiled executable and resource profiles alone cannot establish the exact source-level CFG; debugging symbols, instrumentation, or disassembly may be needed.

---

6. Mobile Payment Gateway – State Transition Testing

Question:
A mobile payment gateway provides JSON API responses for Pending, Success, Reversal, and Timeout states. Reconstruct the state machine, identify transition testing, and develop tests for legal and illegal transitions.

Answer:

1. Understanding the System

A payment gateway processes transactions through different states depending on the payment response, bank confirmation, network timeout, and reversal request.

2. System Decomposition

- Request Validation: Validates transaction ID and amount.
- Payment Processor: Sends the payment request.
- State Manager: Maintains the current transaction state.
- Response Handler: Processes success, failure, or timeout responses.
- Reversal Handler: Handles reversal and reconciliation.
- Database: Stores transaction status and history.

3. State Transition Diagram

Initiated → Pending.

From Pending:

- Confirmation → Success.
- Network delay → Timeout.

- Permitted reversal request → Reversal.

From Success:

- An authorized reversal may lead to Reversal, depending on the payment provider's rules.

The actual legal transitions depend on the payment provider's API specification.

4. State Transition Test Cases

Test Case	Transition or Event	Expected Result
TC1	Initiated → Pending	Transaction accepted for processing
TC2	Pending → Success	Payment confirmed
TC3	Pending → Timeout	Timeout recorded
TC4	Success → Reversal	Reversal processed if permitted
TC5	Success → Success again	Duplicate processing prevented
TC6	Invalid state value	Request rejected
TC7	Timeout followed by delayed success	Reconciliation prevents double debit
TC8	Repeated reversal request	No duplicate refund

5. Testing Technique

State transition testing checks the system's response to changes from one state to another. It covers valid transitions, invalid transitions, state sequences, and repeated events.

6. Improved Strategy

Validate every permitted transition, reject prohibited transitions, and test duplicate callbacks, delayed responses, network interruptions, and inconsistent JSON fields. Confirm that each transaction has a consistent final status and that funds are not debited or refunded twice.

Conclusion: State transition testing ensures reliable payment processing and prevents duplicate transactions, incorrect reversals, and inconsistent transaction states.

---

7. Encrypted Communication Module – Fuzzing and BVA

Question:
An encrypted communication module exposes input parameters and encrypted outputs. Use fuzzing to identify validation boundaries, decompose the module, and reconstruct test cases using BVA to prevent buffer overflow and injection vulnerabilities.

Answer:

1. Understanding the System

The encrypted communication module accepts input data, validates its size and format, encrypts the data, and produces encrypted output. Invalid or oversized inputs may cause crashes, memory corruption, or security vulnerabilities.

2. System Decomposition

- Input Handler: Receives user data.
- Validation Module: Checks length, type, format, and allowed characters.
- Encryption Module: Encrypts validated data.
- Output Handler: Returns ciphertext and error messages.
- Error Handler: Safely processes invalid inputs.

3. Boundary Value Analysis

Assume the application accepts plaintext messages up to 1,024 bytes.

Test Case	Input Length	Expected Result
TC1	0 bytes	Accept or reject according to specification
TC2	1 byte	Process valid input
TC3	1,023 bytes	Process valid input
TC4	1,024 bytes	Accept maximum permitted length
TC5	1,025 bytes	Reject safely
TC6	Very large input	Reject or limit resources safely

4. Fuzzing Tests

Fuzzing supplies unexpected, malformed, or random inputs to detect failures. Test null bytes, invalid encodings, long strings, malformed parameters, and unexpected data types. Use only authorized test environments.

5. Security Validation

- Validate input length before copying data into buffers.
- Use safe memory-handling functions and bounds checks.
- Reject malformed data without crashing.
- Use parameterized operations where inputs reach databases or command interpreters.
- Ensure errors do not expose keys, plaintext, or sensitive information.
- Verify encryption and decryption correctness using known test vectors.

6. Improved Strategy

Combine BVA with fuzzing, invalid-input testing, and memory-safety analysis. Record crashes, unexpected outputs, and resource exhaustion. Retest every discovered defect after fixing it.

Conclusion: BVA identifies failures near input limits, while fuzzing discovers unexpected failures. Together, they improve robustness and reduce buffer overflow and injection risks.

---

8. Insurance Premium Calculator – Branch and Path Coverage

Question:
An insurance premium calculator achieves 100% statement coverage but misses defects in multi-condition if statements. Identify the original approach's limitations, construct a CFG, and develop tests for branch and path coverage.

Answer:

1. Understanding the System

The calculator determines insurance premiums based on customer age, driving history, and vehicle type. Incorrect combinations of conditions can produce incorrect premium values even when every statement executes.

2. Sample Program

```
def calculate_premium(age, safe_driver):
    if age >= 25 and safe_driver:
        return 5000
    else:
        return 8000
```

3. Control Flow Graph

Start → Check whether age ≥ 25 AND safe_driver.

- If true, set the premium to ₹5,000.
- If false, set the premium to ₹8,000.

Both branches then proceed to the End node.

4. Independent Paths and Test Cases

Test	Age	Safe Driver	Expected Premium
TC1	30	True	₹5,000

TC2	30	False	₹8,000
TC3	20	True	₹8,000
TC4	20	False	₹8,000
TC5	25	True	₹5,000
TC6	24	True	₹8,000

5. Coverage Evaluation

Statement coverage confirms that each statement runs at least once. It does not guarantee that each decision outcome or each individual condition has been adequately tested.

Branch coverage requires both true and false outcomes of the overall decision. Condition coverage requires each atomic condition to evaluate to both true and false. Modified Condition/Decision Coverage (MC/DC) can further demonstrate that each condition independently affects the decision outcome.

6. Improved Strategy

Use the six test cases to cover both decision outcomes, age boundaries, and combinations of the two conditions. For complex calculators, construct a CFG, identify independent paths, and test additional conditions such as driving history and vehicle type.

Strictly speaking, 100% path coverage is generally impossible for programs with unbounded loops, and exhaustive combinations may be impractical. For this loop-free example, the two feasible decision branches can both be covered.

Conclusion: Branch, condition, and path-oriented tests reveal defects missed by statement coverage and improve premium calculation accuracy.

---

9. Cloud File Synchronization – ECP, BVA, and Stress Testing

Question:
A cloud-based file synchronization tool provides user activity logs and lost-file defect reports. Analyze defects related to file size, type, and network speed. Infer the original ECP strategy and reconstruct testing using BVA and stress testing.

Answer:

1. Understanding the System

The synchronization tool uploads, downloads, and updates files between local devices and cloud storage. Data loss can occur because of network interruptions, file-size limits, unsupported formats, or synchronization conflicts.

2. System Decomposition

- File Scanner: Detects new and modified files.
- File Validator: Checks file type, size, and permissions.
- Upload Manager: Transfers files to the cloud.
- Synchronization Engine: Compares local and cloud versions.
- Conflict Handler: Resolves simultaneous updates.
- Recovery Manager: Resumes interrupted transfers and restores missing files.

3. Equivalence Class Partitioning

Parameter	Valid Class	Invalid or Exceptional Class
File size	Within supported limit	Exceeds maximum size
File type	Supported format	Unsupported format
Network speed	Stable connection	Slow or disconnected network
File state	New or modified file	Conflicting or inaccessible file

4. Boundary Value Analysis

Assume the maximum supported file size is 100 MB.

Test Case	File Size	Expected Result
TC1	0 MB	Handle empty file according to specification
TC2	1 MB	Upload successfully
TC3	99 MB	Upload successfully
TC4	100 MB	Accept maximum supported size
TC5	101 MB	Reject or report size limit
TC6	Large file with interrupted network	Resume safely without corruption

5. Stress Testing

Test many simultaneous uploads, repeated network disconnections, low bandwidth, disk-space shortages, and concurrent edits on multiple devices. Verify that file hashes match after transfer and that retries do not create duplicate or corrupted files.

6. Improved Testing Strategy

Combine ECP for supported and unsupported files, BVA for size limits, and stress testing for resource and network constraints. Include recovery tests that interrupt transfers at different stages and verify eventual synchronization.

Conclusion: These techniques improve synchronization reliability, reduce file loss, and ensure correct recovery during failures and heavy workloads.

---

10. E-commerce Search and Filter – Decision Table Testing

Question:
SQL queries from an e-commerce search and filter module are provided without source code. Analyze AND/OR/NOT conditions and reconstruct a decision-table strategy covering search combinations.

Answer:

1. Understanding the System

The search module retrieves products based on search keywords, category, price range, availability, and other filters. The SQL queries reveal how these conditions are combined to produce results.

2. System Decomposition

- Keyword Search: Matches product names or descriptions.
- Category Filter: Selects products from a chosen category.
- Price Filter: Applies minimum and maximum price limits.
- Availability Filter: Selects products currently available.
- Query Builder: Combines conditions using AND, OR, and NOT.
- Result Handler: Returns matching products and handles empty results.

3. Decision Conditions

Assume the user selects a category, price range, and availability filter.

- C1: Product belongs to the selected category.
- C2: Product is within the selected price range.
- C3: Product is available.

The decision rule is:

R = C1 AND C2 AND C3

A product is returned only when all three conditions are true.

4. Decision Table

Rule	C1: Category	C2: Price	C3: Available	Expected Result

R1	True	True	True	Include product
R2	False	True	True	Exclude product
R3	True	False	True	Exclude product
R4	True	True	False	Exclude product
R5	False	False	True	Exclude product
R6	False	True	False	Exclude product
R7	True	False	False	Exclude product
R8	False	False	False	Exclude product

5. Improved Test Cases

Test each decision-table rule using a known dataset. Add tests for empty keywords, minimum and maximum prices, missing filters, special characters, duplicate products, and no matching results. Verify that AND returns only products satisfying all selected conditions, while OR returns products satisfying at least one applicable condition.

6. Completeness Evaluation

Three Boolean conditions produce $2^3 = 8$ possible combinations, all represented in the table. However, additional tests are required for NOT operations, null values, invalid parameters, SQL injection attempts, and interactions between optional filters.

Conclusion: Decision-table testing systematically covers combinations of search conditions and detects errors in complex filtering logic. Combining it with boundary testing and security testing improves the completeness and correctness of the search module.

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation